LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ***??***

SUMMARY Authentication does not guarantee compliance with a receiver's contact policy. We identify an order-dependent SAGA matcher defect converting specified denial into permission. Controlled Provider/receiver executions confirm propagation to token issuance and storage. In an evidence-linked symbolic scenario, ProVerif reports conditional `ChatAccept` reachability while the prior-`TokenIssue` correspondence holds, separating token origin from policy compliance.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

LLM-based agents increasingly use external tools and services [1], [2]. Authentication establishes identity; receiver-side authorization determines whether the identified agent may communicate [3]. Authentication and token secrecy leave policy-compliant authorization as a separate security requirement.

SAGA [4] uses contact policies and access-control tokens to govern communication between registered agents. Its production matcher implements the receiver's policy decision. We investigate authorization decision consistency and its security consequences by asking: *Can an implementation-level mismatch with the receiver's intended contact policy authorize a denied agent, and how far can the resulting permission propagate through the communication protocol?*

Our evidence-linked analysis combines independent specification decision comparison, provenance-bound formal consequence analysis, and controlled consumer-path validation. Production matcher observations reveal an order-dependent inconsistency with the intended most-specific matching semantics. An authorization coverage diagnostic explains why SAGA's existing authentication and token-secrecy results leave this requirement unestablished; it does not identify the implementation defect.

The recorded policy denies while the matcher permits. This permission reaches token issuance and storage through controlled Provider and receiver paths. Under the corresponding fixed symbolic divergence, `ChatAccept` is reachable while its prior-`TokenIssue` correspondence holds. These results distinguish policy-compliant authorization from authenticated identity and consistent token origin; implementation and symbolic endpoints remain separate evidence layers.

Our contributions are (1) identification of an authorization decision inconsistency, explaining the matcher's deviation from most-specific semantics and confirming the recorded Deny/Allow counterexample with production observations; (2) evidence-linked formal consequence analysis using finite specification evaluation, provenance checks, and constrained scenario reconstruction to analyze conditional `ChatAccept` reachability and token-origin correspondence; and (3) controlled consumer-path validation of propagation to token issuance and storage through production Provider and receiver code with explicit fixtures and controls.

(1) Related work.

Prior work studies authorization languages and policy semantics [5], [6] and policy verification and change-impact analysis [7]. SPKI [3] and Macaroons [8] provide credential- and capability-based authorization abstractions. ProVerif and Tamarin support symbolic protocol verification [9], [10]. Our case analysis connects an observed implementation deviation from policy semantics to its consumer and symbolic protocol consequences.

2. Authorization Decision Inconsistency

(1) Intended authorization semantics.

SAGA's Provider handles contact requests under the receiver's ordered policy, which maps identity patterns to integer budgets. The Provider and receiver independently call the production matcher: the Provider gates one-time key (OTK) release, and the receiver rechecks authorization before token issuance.

SAGA, Section IV-D.1 [4], specifies selection of the most-specific matching rule; `aid_specificity()` scores the patterns. A positive budget permits the request; a negative budget denies it. We consider fixed policies and valid identities, excluding equal-specificity conflicts and zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`. The artifact binds the archived matcher observation to pinned upstream source provenance.[†] Its `best_pattern` starts at `None`, whose specificity is -1. A more-specific matching rule updates the budget but never `best_pattern`. Later matches still compare against -1,

[†]TODO: affiliation and postal address.

DOI: 10.1587/transfun.E0.A.1

[†]Reproduction artifact: `github.com/g1020268262-a11y/SAGA-Authorization-Analysis-Artifact`.

Copyright © 200x The Institute of Electronics, Information and Communication Engineers

allowing a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and * as 0; their budgets are −1 and 10, respectively.

Table 1 Specification and matcher budgets for two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	−1	10
Wildcard allow, then specific deny	−1	−1

The independent finite specification evaluator selects denial for both orders in Table 1. Specific deny followed by wildcard allow changes the specified −1 into an observed 10; reversing the order restores denial. This counterexample establishes an implementation-level authorization decision inconsistency for the recorded policy, with permission dependent on rule order.

3. Formal Authorization Analysis

(1) Authorization and verification coverage.

Formal consequence analysis distinguishes token origin from policy-compliant issuance. In the consequence model, the verified non-injective correspondence is

$$\mathsf{ChatAccept}(R, I, t, m) \Longrightarrow \mathsf{TokenIssue}(R, I, t)$$

where R is the receiver and token issuer, and I is the initiator. Prior issuance must match R, I, and encrypted token t. The distinct ideal authorization condition for a static receiver policy is

$$\mathsf{TokenIssue}(R, I, t) \Longrightarrow \mathsf{Allow}_{\mathrm{spec}}(P_R, I),$$

where P_R is R's contact policy and $\mathsf{Allow}_{\mathrm{spec}}$ denotes permission under its intended semantics, rather than the matcher result. This second formula is a desired condition, neither an executed ProVerif query nor a proved theorem.

The archived ProVerif 2.05 diagnostic preserves SAGA's original authentication and token-secrecy results but reports an added prior-`PolicyAllow` correspondence for `Accept` as false. Here `Accept` denotes token receipt and verification, distinct from `ChatAccept`. `PolicyAllow` is not connected to actual policy enforcement. The failure exposes a proof-coverage gap; it neither executes the Python matcher nor independently identifies its defect.

(2) Evidence-linked scenario construction.

The Decision Bridge independently evaluates the recorded policy and checks the archived production matcher observation against the ordered rules and initiator identity. It verifies observation hashes and historical source binding before forming the checked Deny/Allow decision pair. That pair reconstructs only the designated scenario region of a hash-pinned ProVerif template, preserving its protocol semantics and queries. This makes the transfer of recorded decision evidence checkable and reproducible.

The Bridge supports only the finite policy grammar needed for this case. It proves neither evaluator correctness for arbitrary policies nor Python-to-ProVerif refinement, and is not a general program translator. ProVerif takes the checked divergence as scenario input; it does not execute Python or the production matcher. Only authorization decisions and token-lifecycle events link the implementation and symbolic evidence.

(3) Symbolic consequences.

The initiator has a valid registered identity, follows the prescribed symbolic peer process, and cannot alter the receiver's fixed policy. The Provider and Certificate Authority (CA) are trusted, cryptographic checks are effective, and private keys are uncompromised. A private symbolic channel abstracts authenticated, confidential transport; reachability is not an arbitrary-adversary claim.

`TokenIssue`(R, I, t) follows `ActiveToken` storage of R, I, encrypted token t, and I's public access-control key, and precedes transmission. For private-channel message m, `ChatAccept`(R, I, t, m) occurs only after checking the fixed identity and an `ActiveToken` entry matching R, I, t, and I's sender public key.

Under the fixed Deny/Allow divergence, the archived run reports `ChatAccept` reachable and, separately, the token-origin correspondence true. Its witness includes issuance; no standalone `TokenIssue` reachability query exists. Token-origin consistency thus coexists with intended-policy denial. Symbolic message acceptance and controlled consumer execution remain separate evidence layers, not a complete service-level end-to-end attack.

4. Consumer-Side Propagation Validation

The validation asks whether incorrect permission survives the production consumer checks and reaches token issuance and storage. The controlled harness drives the Provider `/access` route and receiver handshake, including OTK release and consumption. Archived source hashes bind the matcher, consumers, and token-related cryptographic code to the pinned upstream baseline.

Signed registration material, local certificates, and in-memory transport provide fixtures while production cryptographic checks remain active. Both consumers call the original matcher; wrappers record and return its result unchanged. Fixtures preset neither authorization decisions nor token state.

(1) Control design.

Both consumers use identical policies. D/A places specific deny (−1) before wildcard allow (10); A/A makes both budgets 10, and D/D reverses the D/A order. The letters denote specification and observed implementation decisions, respectively. A/A tests the permitted-token path, and D/D

tests Provider-side rejection, providing controls with agreeing decisions. Each case ran once with fresh state and one OTK.

Table 2 Production consumer results for the divergent D/A case.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider `/access`	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

In D/A, Table 2 confirms that permission from both production matchers reaches token issuance and `active_tokens` storage despite specification denial. One token is generated, sent, and stored. A/A also reaches token sending and storage; D/D returns HTTP 403 without OTK release or state change and never reaches the receiver, so receiver rejection remains untested.

These observations establish controlled consumer propagation, not application message acceptance. No application message is sent: a boundary stub intercepts `receive_conversation` before the original handler executes.

5. Scope and Conclusion

For the recorded SAGA policy, order-dependent matching authorizes an agent denied by the intended most-specific semantics. The resulting permission reaches token issuance and storage through controlled production consumers; the corresponding fixed symbolic model permits `ChatAccept` while its prior-`TokenIssue` correspondence holds. Authentication and token-origin consistency are insufficient to establish compliance with the receiver's intended authorization policy.

These findings concern the recorded case, tested policies, fixtures, and symbolic assumptions. They establish neither arbitrary-policy correctness nor Python-to-ProVerif refinement or an arbitrary-attacker or deployed exploit. Consumer execution stops at token issuance and storage; message acceptance belongs only to the symbolic model.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W.X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol.18, no.6, p.186345, 2024.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," Advances in Neural Information Processing Systems 37 (NeurIPS 2024), pp.82895–82920, 2024.

[3] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, "SPKI certificate theory," RFC 2693, RFC Editor, Sept. 1999. Experimental.

[4] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," Network and Distributed System Security (NDSS) Symposium, Internet Society, Feb. 2026.

[5] M.Y. Becker and P. Sewell, "Cassandra: Distributed access control policies with tunable expressiveness," 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY), 2004.

[6] M.Y. Becker, C. Fournet, and A.D. Gordon, "Design and semantics of a decentralized authorization language," 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.

[7] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, "Verification and change-impact analysis of access-control policies," 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.

[8] A. Birgisson, J.G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," Network and Distributed System Security Symposium (NDSS), Internet Society, 2014.

[9] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.

[10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," Computer Aided Verification (CAV), Lecture Notes in Computer Science, vol.8044, pp.696–701, Springer, 2013.